PayLedger – project memory

What this is

A merchant payments and wallet REST API for a FinTech portfolio project.
Customers pay merchants from wallets; every money movement is recorded in a
double-entry ledger. Correctness under concurrency matters more than features.

Stack (do not change without asking)

- Java 17, Spring Boot 3 (latest stable), Maven wrapper (./mvnw)
- PostgreSQL 16 via docker-compose; Flyway migrations; Spring Data JPA
- Spring Security with JWT (stateless); BCrypt for passwords
- JUnit 5, Spring Boot Test, Testcontainers (real Postgres in tests, never H2)
- springdoc-openapi for Swagger UI

Money rules (never break)

- Store amounts as BIGINT in paise (minor units). Never float or double.
- Never update a balance column. Balance = SUM of ledger_entries for a wallet.
- Every money movement writes entries that sum to zero, in ONE transaction.
- Lock wallets with SELECT ... FOR UPDATE in ascending wallet id order.

Code conventions

- Package root: com.payledger; layers: controller -> service -> repository.
- DTOs (Java records) at the API boundary; never expose JPA entities.
- Errors: one @RestControllerAdvice returning {code, message, details}.
- Validate input with jakarta.validation annotations.
- Never edit an applied Flyway migration; add a new V__name.sql file.

Commands

- Run DB: docker compose up -d
- Build + all tests: ./mvnw verify
- Run app: ./mvnw spring-boot:run

Working rules for Claude

- Plan first for any multi-file change; wait for approval.
- Every feature ships with tests in the same change. Run ./mvnw verify and show the result before saying a task is done.
- Never use @Disabled, skip tests, or weaken an assertion to make a test pass.
- After each task, summarise: files changed, why, and how to verify.
- Explain non-obvious decisions in 1-2 lines; I must be able to defend them in an interview.